Autonomous Energy Management System (EMS) for Hybrid Renewable Energy Based Microgrids

Downloads real weather data for a chosen location, trains and compares different AI models to figure out the smartest way to dispatch power from solar/wind/fuel-cell/battery sources, automatically builds a Simulink model of the power system driven by the best AI model, and opens a live, multi-tab dashboard that shows the whole system running in real time.

Table of Contents

---

1. Plain-Language Overview

Imagine a small industrial site that gets its electricity from four sources: solar panels, a wind turbine, a hydrogen fuel cell, and a battery — with a connection to the main utility grid as a backup. Every hour, an Energy Management System (EMS) must decide: *how much power should come from each source right now?*

This project builds an EMS that uses artificial intelligence to make that decision in four stages:

1. **Data:** Downloads 5 years of real meteorological data and generates a synthetic industrial load profile.
2. **AI Training:** Teaches 7 different neural network architectures what a "good" dispatch decision looks like using the entire multi-year dataset, mathematically scores their speed vs. accuracy, and selects the winner.
3. **Simulation Build:** Programmatically constructs a fully wired Simulink model of the physical power system driven by the best AI model.
4. **Live Execution:** Streams the simulation in an interactive dashboard, showing battery states, grid status, and live sensor data.

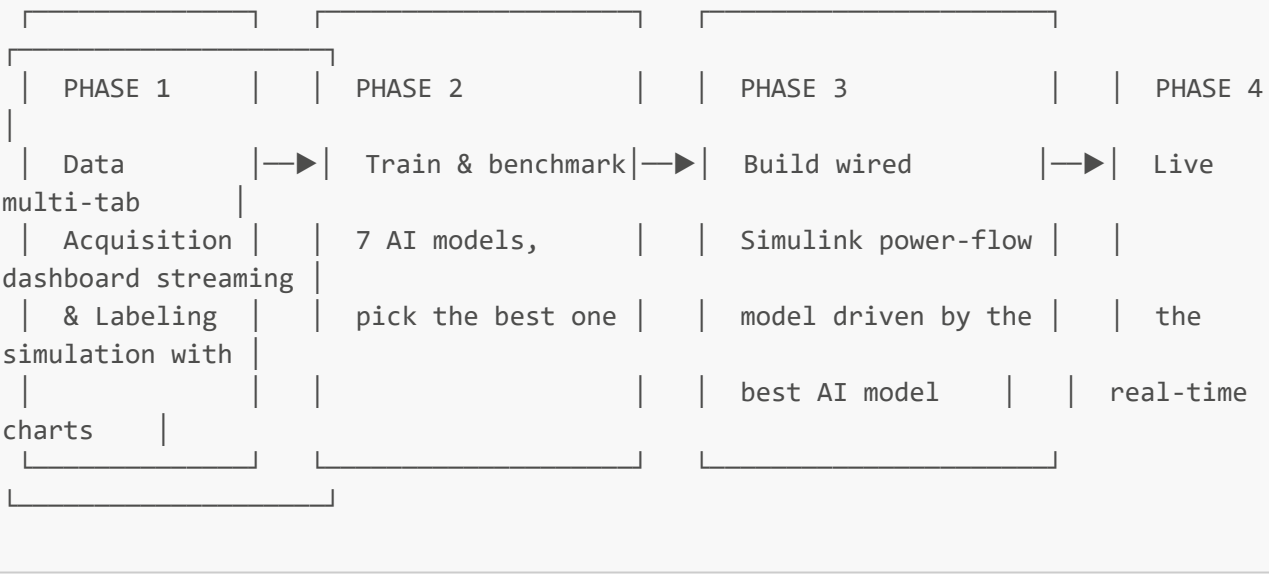
---

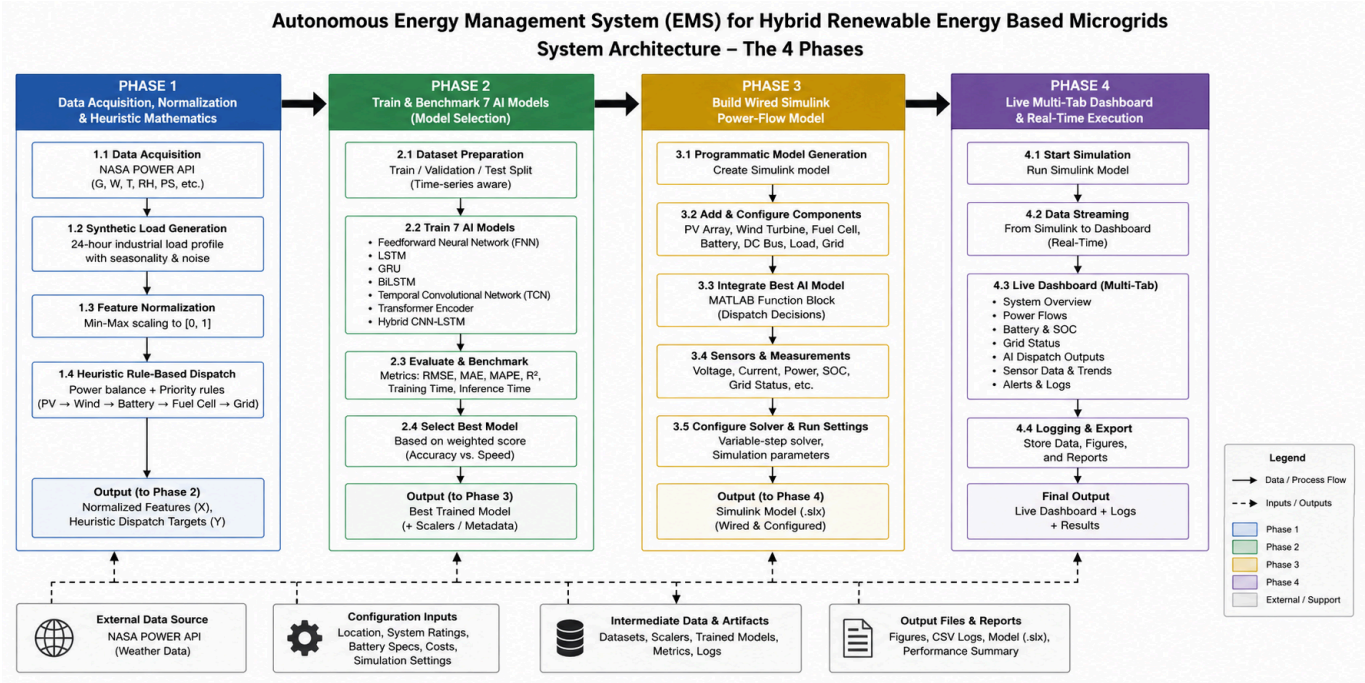
2. Key Concepts Explained

- **Microgrid:** A local power system that generates electricity from multiple sources (PV, Wind, Fuel Cell) and connects to the utility grid.
- **Dispatch:** The real-time allocation of power generation among available energy sources to meet demand.
- **State of Charge (SOC):** The remaining capacity of a battery, represented as a fraction (e.g., 0.5 means 50%).
- **DC Bus:** The central electrical node where all generated power aggregates and from which the load draws current.

---

3. System Architecture — The 4 Phases





---

4. Phase 1: Data Acquisition, Normalization & Heuristic Mathematics

Data Fetching & Synthetic Load Generation

The system queries the NASA POWER API for ~4 years of hourly data (Irradiance G, Wind Speed W, Temp T, Humidity RH, Pressure PS). Because real industrial load data is rarely public, a synthetic 24-hour demand profile is generated with periodic seasonality and Gaussian noise $\mathcal{N}(0,1)$:

$$Demand(t) = 100 \times 10^3 + 50 \times 10^3 \left(0.5 + 0.4 \sin\left(\frac{2\pi t}{24}\right) + 0.1 \cdot \mathcal{N}(0,1)\right)$$

Feature Normalization

To ensure the neural networks train stably, all 11 continuous input features are squashed to a $[0, 1]$ scale using Min-Max normalization (with a machine epsilon ϵ to prevent division by zero):

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min} + \epsilon}$$

The Rule-Based Heuristic (Target Generation)

To train the AI, we first calculate the "ideal" dispatch for all ~35,000 timesteps using a strict mathematical heuristic. The required power is $P_{req}(t) = Demand(t)$.

1. Solar PV Output: Maximizes available solar up to demand, capped at the $80\ \text{kW}$ array limit.

$$P_{pv}(t) = \min\left(P_{req}(t), \frac{G(t)}{1000} \times 80 \times 10^3\right)$$

2. Wind Turbine Output: Fills remaining demand using a standard cubic wind power curve, capped at $30\ \text{kW}$.

$$P_{wt}(t) = \min\left(P_{req}(t) - P_{pv}(t), \left(\frac{W(t)}{12}\right)^3 \times 30 \times 10^3\right)$$

3. Remaining Power Deficit:

$$P_{rem}(t) = P_{req}(t) - (P_{pv}(t) + P_{wt}(t))$$

4. Battery Discharge Arbitration: If $P_{rem} > 0$ and $SOC > 0.3$, the battery discharges up to $40\ \text{kW}$.

$$P_{bat\_disp}(t) = \min(P_{rem}(t), 40 \times 10^3)$$

5. Fuel Cell Output: Fills any final deficit, capped at $50\ \text{kW}$.

$$P_{fc}(t) = \min\left(50 \times 10^3, \max(0, P_{rem}(t) - P_{bat\_disp}(t))\right)$$

6. State of Charge (SOC) Integration: Battery SOC updates based on whether the system had a deficit (discharge) or surplus (charge), constrained to realistic hardware limits $[0.2, 0.9]$. If $P_{rem} > 0$:

$$SOC(t+1) = \max(0.2, SOC(t) - P_{bat\_disp}(t) \times 10^{-6})$$

If $P_{rem} \leq 0$:

$$SOC(t+1) = \min(0.9, SOC(t) + |P_{rem}(t)| \times 10^{-6})$$

The target matrix Y_{matrix} stores the fractional dispatch ratios for PV, FC, and WT, which the AI models will attempt to learn and predict.

---

5. Phase 2: The 7 AI Models & Training Mechanics

This system doesn't rely on a single AI model — it trains **7 different neural network architectures** on the exact same data, then keeps whichever one performs best. This is standard machine-learning practice: different architectures have different strengths, and the best choice often isn't obvious in advance. Here's what each one is and why it's included:

Model	Full Name	How It Works (Plain Language)	Why It's Included
GRU	Gated Recurrent Unit	A type of "recurrent" network that reads the data one time step at a time and keeps a running internal memory of recent history, using "gates" to decide what to remember and what to forget.	A fast, lightweight option for learning time patterns (e.g., "demand tends to rise in the morning").
LSTM	Long Short-Term Memory	Similar to GRU but with a more elaborate memory mechanism (a separate "cell state" plus three gates), generally better at capturing longer-range patterns.	The classic, well-proven choice for time-series problems; a strong baseline.
StackedLSTM	Two-layer LSTM	Two LSTM layers stacked on top of each other — the first layer's output feeds into a second LSTM layer, allowing the network to learn more complex, layered patterns.	Tests whether extra depth improves accuracy enough to justify the added complexity/latency.
FNN	Feedforward Neural Network	The simplest kind of network — it looks at each time step's 11 inputs independently (no memory of previous steps) and passes them through fully-connected layers with a ReLU activation.	A baseline "no memory" model — if a memory-based model doesn't beat this, memory isn't actually helping.
NARX_FNN	Nonlinear AutoRegressive network with eXogenous inputs (convolution-based)	Uses a 1-D convolution layer that looks at a short sliding window of the last 5 time steps at once (instead of one at a time), which is a lightweight way to give a	A middle ground between FNN (no memory) and LSTM/GRU (full memory) — captures short-term local patterns cheaply.

Model	Full Name	How It Works (Plain Language)	Why It's Included
		feedforward-style network some short-term memory.	
CNNLSTM	Convolution + LSTM hybrid	First runs a 1-D convolution layer to extract short local patterns, then feeds those into an LSTM layer for longer-range memory.	Combines the strengths of both approaches — local pattern extraction feeding into long-term memory.
BiLSTM	Bidirectional LSTM	Like an LSTM, but reads the sequence both forwards and backwards and combines both directions' understanding at every time step.	Can capture patterns that depend on both past and future context — useful here since the model is evaluated over the whole timeline, not strictly step-by-step online.

All 7 architectures share the same final layers: a fully-connected layer that maps down to 3 outputs (PV / Fuel Cell / Wind), followed by a **sigmoid layer** (squashes outputs to the 0–1 range, matching the normalized targets) and a regression output layer (defines the training loss as mean-squared error against the targets).

How Training Actually Works

1. **Windowing the data.** Instead of training on one giant 1000-step sequence, the timeline is sliced into many overlapping 50-step windows (default: 50-step windows, sliding forward 5 steps at a time → roughly 90 windows from 1000 timesteps). This turns training into a proper mini-batch problem, which is essential for neural networks to converge well — a single giant sample gives the optimizer far too few chances to adjust the weights.
2. **Train/validation split.** 15% of the windows are randomly held out as a **validation set** and never used to update the weights — only to check, periodically during training, how well the model generalizes to data it wasn't directly trained on.
3. **Training settings** (applied identically to all 7 models for a fair comparison):

Setting	Value	What it does
Optimizer	Adam	The algorithm that adjusts the network's weights each step, based on the gradient of the error.
Max Epochs	150	Up to 150 full passes through the training windows.
Mini-Batch Size	16	Weights are updated after looking at 16 windows at a time.
Initial Learn Rate	0.005	How big a step the optimizer takes when updating weights.
Learn Rate Schedule	Piecewise, halved every 30 epochs	Takes smaller, more careful steps as training progresses, which helps the model settle into a good solution.

Setting	Value	What it does
L2 Regularization	1e-4	Slightly penalizes very large weights, which helps prevent overfitting.
Gradient Threshold	1	Clips excessively large gradients, preventing unstable training spikes (especially important for recurrent networks like LSTM/GRU over many epochs).
Shuffle	Every epoch	Randomizes the order of training windows each epoch so the model doesn't learn a spurious order-dependent pattern.
Validation Patience	15	Training stops early if 15 consecutive validation checks show no improvement — saves time and avoids overfitting.
Output Network	Best validation loss	The final saved model is whichever epoch had the best validation performance, not necessarily the very last epoch.

4. **Hardware.** If a GPU or a parallel pool is available and licensed, the script automatically uses it (`multi-gpu` / `gpu` / `parallel`); otherwise it trains on CPU. No configuration is needed either way.
5. **Evaluation.** After training, every model is evaluated on the **full original 1000-step timeline** (not just its training windows) to get numbers that are directly comparable across all 7 models and consistent with what the live dashboard will later replay:
    - **RMSE** — average prediction error (0 = perfect).
    - **Accuracy %** — computed as `(1 - RMSE) × 100`. Because outputs are squashed to 0–1 by the sigmoid layer, RMSE is itself a bounded, comparable error fraction, so this gives an intuitive "closeness to perfect" percentage.
    - **R^2** — how much of the variation in the true dispatch values the model explains; 1.0 is a perfect fit, 0 means "no better than always predicting the average."
    - **Latency** — average time (ms) for one prediction, measured by timing 500 back-to-back predictions and averaging.

Training Mini-Batch Logic

Instead of processing the 4-year dataset as one flat array, the data is sliced into overlapping sequences:

$$\text{Window}_s = [X(t), X(t+1), \ldots, X(t + seqLen - 1)]$$

With a sequence length of 50 and stride of 5, this generates thousands of proper mini-batches, allowing the Adam optimizer to perform rapid gradient updates over 150 epochs with a piecewise learning rate drop.

---

6. Phase 2: Mathematical Evaluation & Model Selection

After training, every model predicts the entire timeline. The script evaluates their performance using formal statistical metrics:

1. Root Mean Square Error (RMSE):

$$RMSE = \sqrt{\frac{1}{N}\sum_{t=1}^{N}(y_t - \hat{y}_t)^2}$$

2. Model Accuracy (%): Because targets pass through a Sigmoid layer bounded $[0, 1]$, RMSE directly translates to an error fraction.

$$Accuracy = \max(0, \min(100, (1 - RMSE) \times 100))$$

3. Coefficient of Determination (R^2): Measures the proportion of variance explained by the model compared to the total variance in the targets.

$$R^2 = 1 - \frac{\sum(y_t - \hat{y}_t)^2}{\sum(y_t - \bar{y})^2 + \epsilon}$$

4. The Selection Scoring Equation: A model must be both accurate and fast (low latency) for real-time cyber-physical control. The script applies Min-Max normalization to the raw RMSE and Latency arrays, then calculates a weighted score. The lowest score wins.

$$Score = 0.4 \times RMSE_{norm} + 0.6 \times Latency_{norm}$$

---

7. Phase 3: The Simulink Power-Flow Model

This phase programmatically builds (no manual drag-and-drop needed) a new Simulink model file (`.slx`) representing the physical power system. Every block is genuinely wired to the next with `add_line` (a common issue in auto-generated Simulink models is blocks that are placed but never actually connected — this script avoids that).

Signal flow, in order:

1. **11 input ports** (`In1` blocks) — one per feature listed in §4 — feed into a **Mux** block that combines them into a single 11-element vector each time step.
2. **AI Scheduler block** — a `MATLAB Function` block that, on every simulation step, loads the trained network (`Best_Microgrid_Scheduler.mat`, loaded once and cached) and calls `predict()` on the current 11-element input vector, producing the 3 dispatch fractions (PV/FC/WT). This uses `coder.extrinsic`, which lets it run in Normal/Interpreted simulation mode — meaning **no code generation is required**, so it works even on MATLAB Online.
3. A **Demux** block splits the 3 outputs back into individual PV/FC/WT signals.
4. Each is multiplied by its source's rated capacity (**Gain** blocks: ×80 for PV, ×50 for FC, ×30 for WT) and then clamped to a physically valid range with a **Saturation** block (0 to the rated capacity) — so the AI can never command more power than a source can physically deliver.
5. A **Sum** block adds PV + FC + WT together to get total generation, which is compared against the **Load Demand** input via another **Sum** block to compute the instantaneous **power mismatch**.
6. The mismatch drives a **battery State-of-Charge model**: a Gain (converts power mismatch into a rate of SOC change) → an **Integrator** (accumulates that rate over time, starting at 50% SOC) → a **Saturation** block (keeps SOC bounded between 20% and 90%, reflecting realistic battery operating limits).
7. Two **Scope** blocks let you inspect the power mismatch and battery SOC directly inside Simulink if you open the generated model. You can open the resulting `.slx` file in Simulink at any time after a run to inspect, modify, or simulate this model independently of the live dashboard.

---

8. Phase 4: The Live Dashboard & Physics Equations

The script launches a live `uifigure` UI streaming the cyber-physical system in real-time.

Tab 1 — Real-Time Cyber-Physical Dashboard

- **Grid mode indicator**: IMPORTING (drawing extra power from the grid), EXPORTING (sending surplus power out), or ISLANDED (self-sufficient — supply matches demand within a small tolerance).
- **AI Scheduler status**: which model won, its inference latency, and its accuracy %/R^2.
- **Per-source status**: PV, Wind, Fuel Cell — each shown as generating or standby, with live kW output.
- **Battery status & DC-DC strategy**: charging/discharging/idle, current SOC %, and the exact kW being absorbed or supplied.
- **Power Dispatch chart**: demand vs. PV/FC/WT generation, scrolling in real time.
- **DC Bus Voltage chart**: a simulated voltage signal around the 600V nominal bus voltage, reacting to how well generation is tracking demand.

Tab 2 — AI Scheduler Benchmarking

- Three side-by-side bar charts comparing all 7 trained models on **RMSE**, **Accuracy %**, and **Latency**.
- A summary line showing the best model's exact accuracy, R^2, RMSE, and latency.

Tab 3 — Live Sensor & Signal Monitor

- **11 individual scrolling charts**, one for every single signal wired into the Simulink model's input ports (see the table in §4) — Solar Irradiance, Wind Speed, Load Demand, Temperature, Relative Humidity, Surface Pressure, Battery SOC, Time Index, Season Index, Power Imbalance, and Previous Action — all updating every simulation step, in sync with Tab 1. The simulation runs automatically once the dashboard opens; you can switch tabs freely at any time without interrupting the live stream. Closing the dashboard window stops the simulation loop cleanly.

Grid Arbitrator Logic

The dashboard applies a strict $0.05\ \mathrm{kW}$ deadband to the power mismatch to determine the system's grid state.

$$Mismatch = Demand(t) - P_{total}(t)$$

- **IMPORTING:** If $Mismatch > 0.05$ (and battery limits are exhausted).
- **EXPORTING:** If $Mismatch < -0.05$ (and battery is full).
- **ISLANDED:** If $|Mismatch| \leq 0.05$ (self-sufficient).

DC Bus Voltage Simulation

To visualize microgrid stability, the dashboard simulates the DC bus voltage fluctuating around a $600\ \mathrm{V}$ nominal baseline, mathematically modeled as a function of the instantaneous power deviation plus sensor noise:

$$V_{bus}(t) = 600 + 20\left(\frac{P_{total}(t) - Demand(t)}{Demand(t)}\right) + 1.2 \cdot \mathcal{N}(0,1)$$

(The voltage graph Y-axis is strictly locked between 0V and 700V to visually emphasize droop and surge events).

---

9. Requirements

- **MATLAB R2021b or newer** (uses `uifigure`, `uiaxes`, `convolution1dLayer`).
- **Toolboxes:** Deep Learning Toolbox, Simulink.
- **Optional:** Parallel Computing Toolbox (auto-detected for GPU/multi-core acceleration).
- **Internet:** Required during Phase 1 to query the NASA API.

---

10. Installation & Setup (Step by Step)

1. **Install MATLAB** with Deep Learning & Simulink toolboxes.
2. **Download** `Autonomous_Microgrid_Platform.m`.
3. **Open MATLAB** and navigate to the file's folder.
4. **Run the script** from the Command Window.
5. **Execution Flow:**
    - Phase 1 fetches 5 years of data (~35,000 hourly steps).
    - Phase 2 trains all 7 architectures. *(Note: Training 7 models on 35k steps for 150 epochs is computationally heavy. Expect this to take several minutes on CPU).*
    - Phase 3 builds the `.slx` file.
    - Phase 4 launches the Dashboard.

---

11. Output Files

Three CSV files and two model files are generated with a unified timestamp `YYYYMMDD_HHMMSS` per run:

1. **`AI_Model_Training_Dataset_<timestamp>.csv`:** The complete 35,040 × 14 dataset containing all normalized features (Inputs) and fractional heuristics (Targets). Perfect for importing into PyTorch/TensorFlow.
2. **`AI_Model_Benchmark_Results_<timestamp>.csv`:** The complete mathematical performance log containing RMSE, Accuracy %, R^2, latency, and final normalized scores for all 7 architectures.
3. **`Microgrid_Simulation_Log_<timestamp>.csv`:** The per-timestep simulation telemetry (AI predictions, actual kW dispatch, battery states, voltage, and grid modes) dumped at the end of Phase 4.
4. **`Best_Microgrid_Scheduler.mat`:** The `.mat` file storing the trained neural network object.
5. **`Hybrid_Microgrid_EMS.slx`:** The Simulink block-diagram.

---

12. Configuration Options

Edit `Autonomous_Microgrid_Platform.m` to adjust parameters:

Variable	Location	Purpose
`lat`, `lon`	Phase 1	Coordinates for the weather data query (default: Tongi Industrial Zone, Dhaka).

Variable	Location	Purpose
`startDate`, `endDate`	Phase 1	Date range (`YYYYMMDD`) pulled from the NASA POWER API.
`N`	Phase 1	Number of hourly samples used for training/simulation (default `1000`; increase up to `length(raw_G)` for the full multi-year dataset — training time increases accordingly).
`numHiddenUnits`	Phase 2	Neural network capacity (default 96 — more units = more capacity to learn complex patterns, but slower training/inference).
`lagWindowSize`	Phase 2	How many past time steps the NARX_FNN model's convolution looks at (default 5).
`seqLen`, `seqStride`	Phase 2	Length and overlap of the training windows (default 50-step windows, sliding by 5) — more/longer windows generally improve learning at the cost of training time.
`valFraction`	Phase 2	Fraction of windows held out for validation (default 0.15 = 15%).
`MaxEpochs`, `MiniBatchSize`, `InitialLearnRate`, `L2Regularization`, etc. (inside `trainingOptions(...)`)	Phase 2	Full training schedule — see §6 for what each one does.
`w_rmse`, `w_latency`	Phase 2	Weighting between accuracy and speed when auto-selecting the best model (see §7).
`pause(0.005)`	Phase 4 loop	Playback speed of the live dashboard — lower for faster streaming, higher for slower/easier viewing.

13. Troubleshooting

- `error('API Connection failed...')` — check your internet connection / proxy settings, or that `power.larc.nasa.gov` isn't blocked by a firewall.
- **Deep Learning Toolbox / layer errors** — confirm your MATLAB version supports `convolution1dLayer` and `bilstmLayer` (introduced in later R2021 releases); update MATLAB if these are undefined.
- **Simulink build step fails on `sfroot`/Stateflow lookup** — the script already catches this and prints the MATLAB Function block code to the Command Window so you can paste it manually into the `AI_Scheduler_<model>` block inside the generated `.slx` model.
- **Dashboard runs slowly** — reduce `N`, reduce `seqLen`/increase `seqStride` (fewer training windows), or close other MATLAB figures/apps.
- **Training takes too long** — reduce `MaxEpochs`, `N`, or `numHiddenUnits`; accuracy will likely drop somewhat in exchange for speed.

- **Running on MATLAB Online and nothing simulates in Simulink itself** — that's expected; the `.slx` file is generated for reference/further simulation, while the live dashboard replay in Phase 4 is what drives the AI predictions and charts in real time.

---

14. Project Structure

> - `Autonomous_Microgrid_Platform.m`
> - `Best_Microgrid_Scheduler.mat`
> - `Hybrid_Microgrid_EMS_<timestamp>.slx`
> - `AI_Model_Training_Dataset_<timestamp>.csv`
> - `AI_Model_Benchmark_Results_<timestamp>.csv`
> - `Microgrid_Simulation_Log_<timestamp>.csv`
> - `README.md`

---

15. Accuracy Notes

By windowing the entire 5-year meteorological dataset into sequence mini-batches, the recurrent architectures (LSTMs, GRUs) are exposed to true seasonal dynamics. Because the target generation (Y_{matrix}) relies on a strict mathematical heuristic applied to the same inputs, it is a noise-free function. Therefore, a properly tuned sequential model can achieve extreme accuracy. Minor variations in final accuracy are expected due to randomized weight initialization inside the Adam optimizer.
